

JAVA 程序设计



授课人：何毅
办公室：1909



第15讲 内容向导

15.1 输入输出流概述

15.2 控制台输入输出

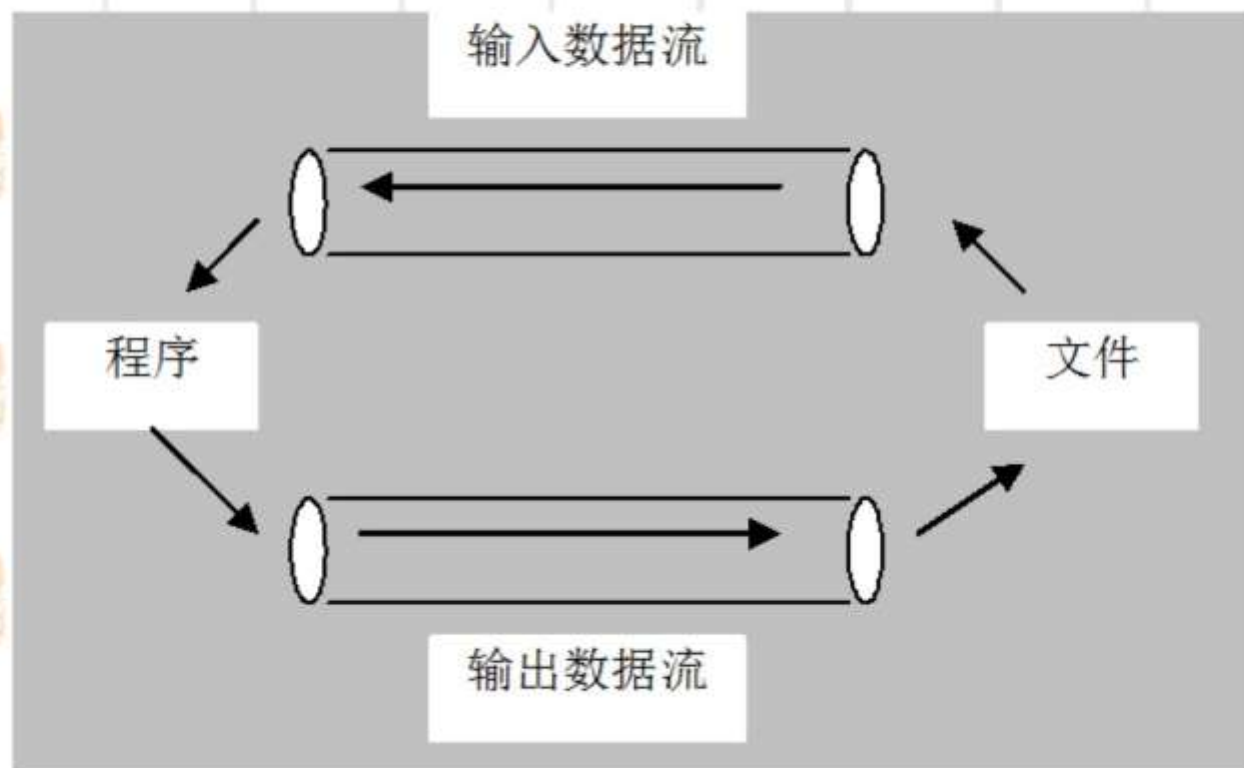
15.3 字节流类



15.1 输入输出流概述

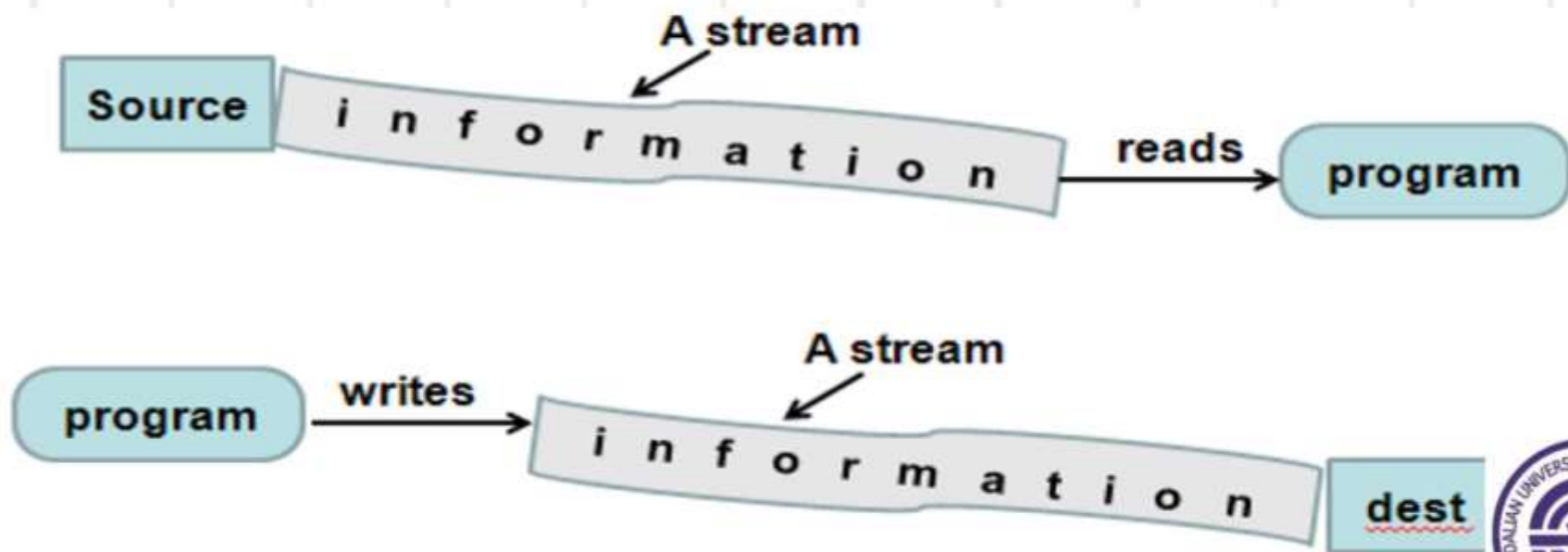
1. 数据流 (Stream)

- 数据流是连续单路流动数据的抽象。在Java中，所有的I/O都是以数据流的形式进行处理的。



➤什么是流

- 流是一个很形象的概念。当程序需要读取数据的时候，就开启一个通向数据源的流，这个数据源可以是文件、内存或是网络连接。类似的，当程序需要写入数据的时候，就会开启一个通向目的地的流。这时候你就可以想象数据在这其中“流”动一样。



➤流的分类

- 按数据的传输方向，分为输入流和输出流。
- 按处理的单位，分为字节流和字符流。
- 按照功能的不同，分为节点流和过滤流。



➤ 流采用缓冲区技术

1. 对流进行读/写操作的最小单位是字节，传输效率低下。
2. 为提高传输速率，可将一块内存空间设计成缓冲区，暂存待传送的数据，这样一次可读/写若干字节。
3. 配备缓冲区的流称为缓冲流。
4. 带缓冲区的输入/输出操作
5. 在缓冲区写满之前若要进行数据传输，称为立即传输（flush）操作。





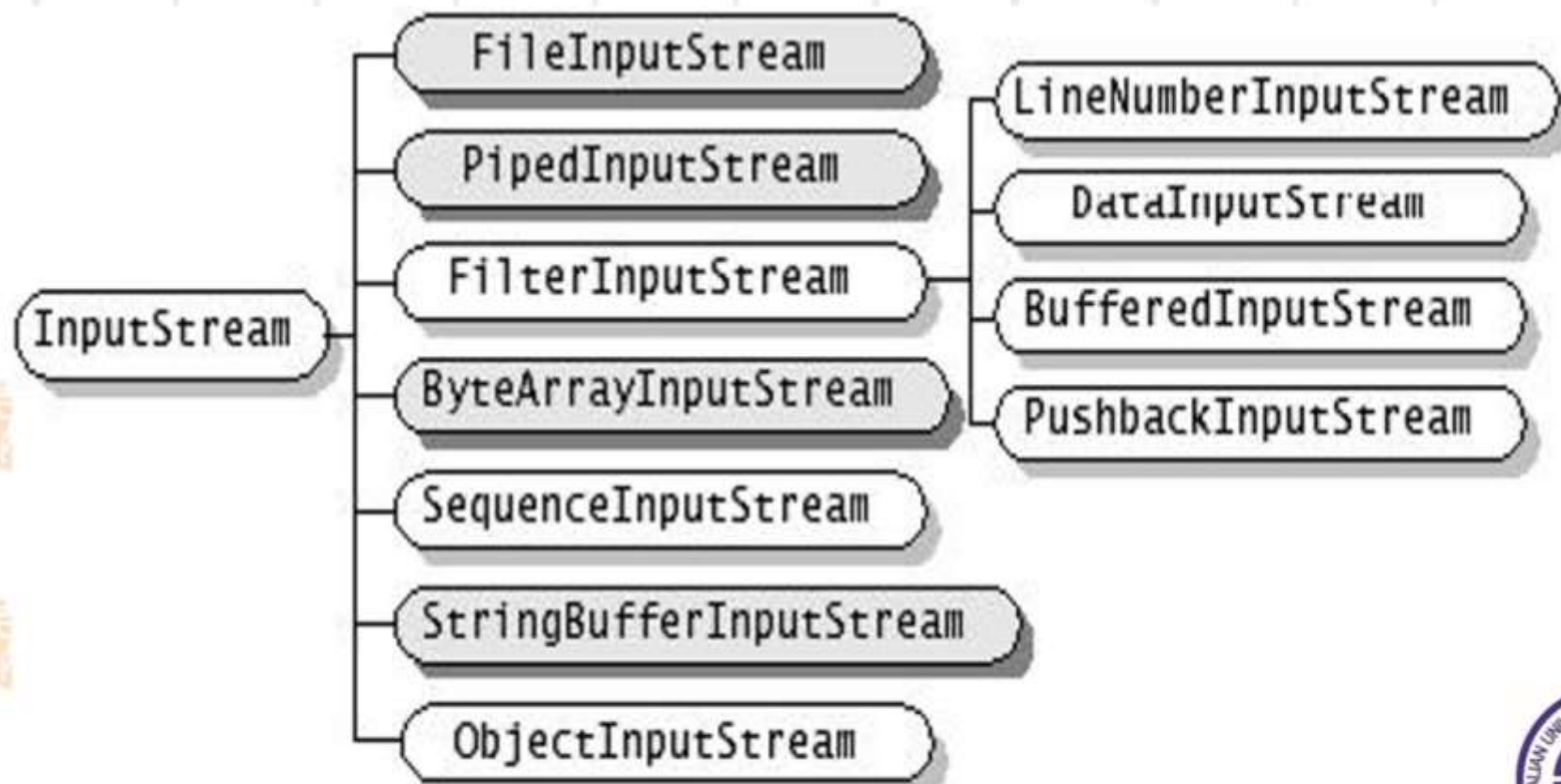
➤流的作用

- 1.设计流的目的是，为了使数据传输独立于相关设备。
- 2.程序中需要根据待传输数据的不同特性而使用不同的流，数据传输给指定设备后的操作由系统执行设备驱动程序完成，这样，程序中不需要关注设备实现细节，使得一个源程序能够用于多种输入/输出设备，从而增强了程序的可重用性。



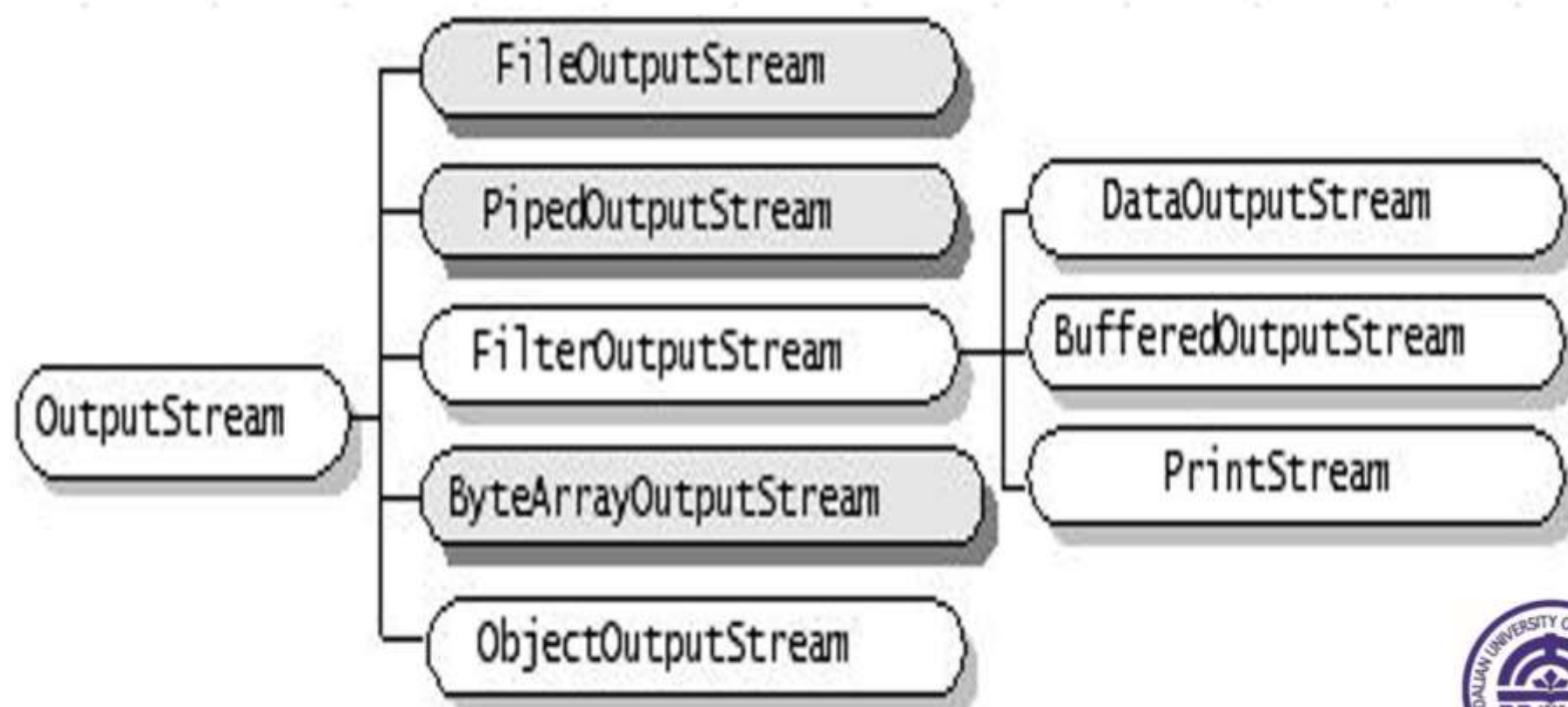
➤ java.io包中的类

1) 字节流InputStream类



➤ java.io包中的类

2) 字节流OutputStream类



15.2 控制台输入输出

➤ 交互式输入输出有两种类型。一种是从键盘进行简单的输入和以纯文本形式进行简单的输出。另一种是在图形环境中进行输入输出。前者称为文本交互式输入输出，后者称为图形交互式输入输出。

➤ 文本输入输出都是使用类 **System**。System 类包含三个 I/O 对象：

(1) **System.in**、**System.out** 和 **System.err**。

(2) 这三个对象是所有 Java 程序员使用的基本对象，用于从键盘输入、向屏幕输出和显示错误信息。System 类存在包 **java.lang** 包中，这个包自动导入到每个类中。



import java.io.*;

```
public class CharIOExample{  
    public static void main(String[]args){  
  
        char c=' '  
        System.out.print("请输入一个字符:");  
        try{  
            c=(char)System.in.read();  
        }catch(IOException e){}  
  
        System.out.println("您输入的字符是:"+c);  
    }  
}
```



➤ Java不直接支持键盘输入。

- **Scanner**是SDK1.5新增的一个类,可是使用该类创建一个对象.
- **Scanner reader=new Scanner(System.in);**
- 然后**reader**对象调用下列方法(函数),读取用户在命令行输入的各种数据类型
- **next.Byte(),nextDouble(),nextFloat,nextInt(),nextLine(),nextLong(),nextShort()**
- 上述方法执行时都会造成堵塞,等待用户在命令行输入数据回车确认



import java.util.*;

```
public class Example{
    public static void main(String args[]){
        System.out.println("请输入若干个数,每输入一个数用回车确认");
        System.out.println("最后输入一个非数字结束输入操作");
        Scanner reader=new Scanner(System.in);
        double sum=0;
        int m=0;
        while(reader.hasNextDouble()){
            double x=reader.nextDouble();
            m=m+1;
            sum=sum+x;
        }
        System.out.printf("%d个数的和为%f\n",m,sum);
        System.out.printf("%d个数的平均值是%f\n",m,sum/m);
    }
}
```

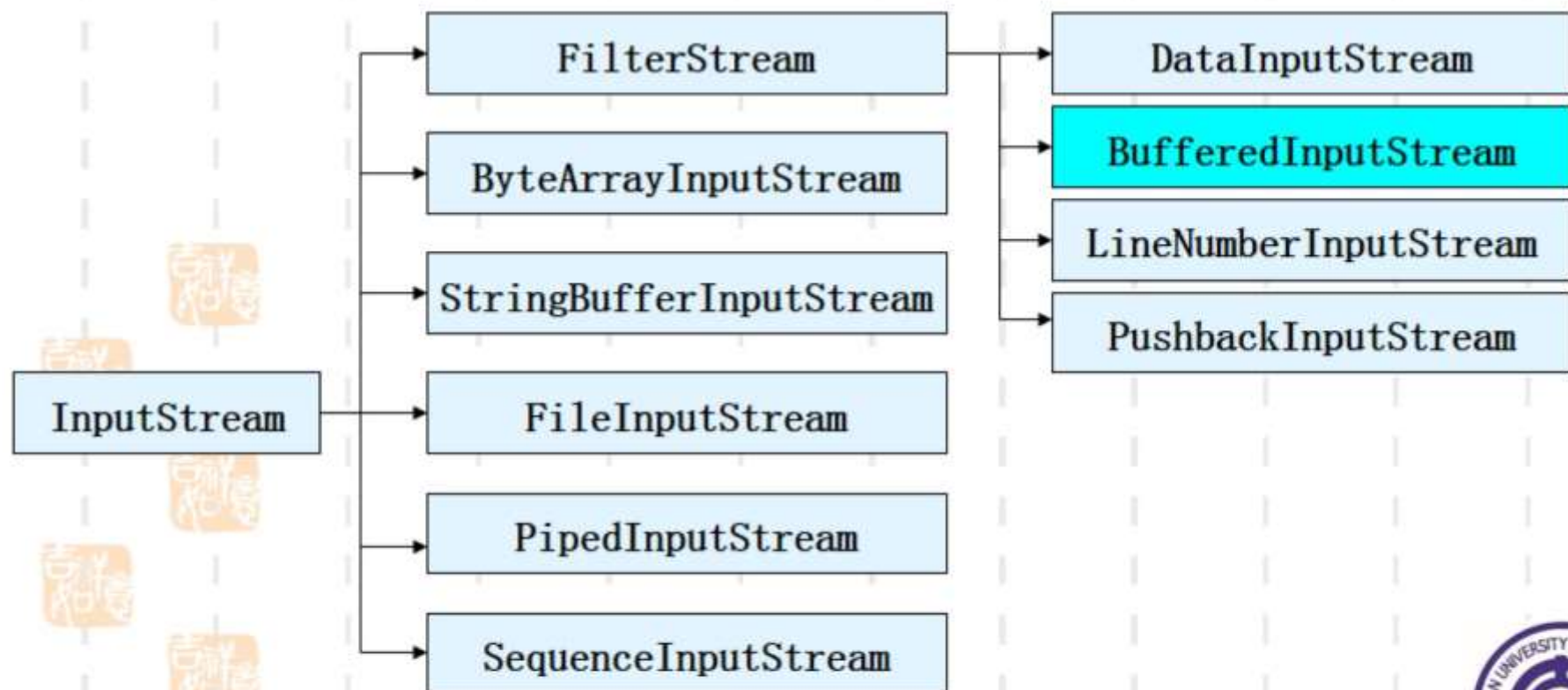


15.3 字节流类

- 字节流每次读写**8位二进制数**，由于它只能将数据以二进制的原始方式读写，而不能分解、重组和理解这些数据，所以可以使之变换、恢复到原来的有意义的状态，因此字节流又被称为**二进制字节流或位流**。
- **InputStream**和**OutputStream**类是抽象类，定义了字节流的基本输入输出操作方法。由于不能创建它们的实例对象，故而它们派生出不同的子类，每个子类实现一种特殊的字节流输入或输出操作。



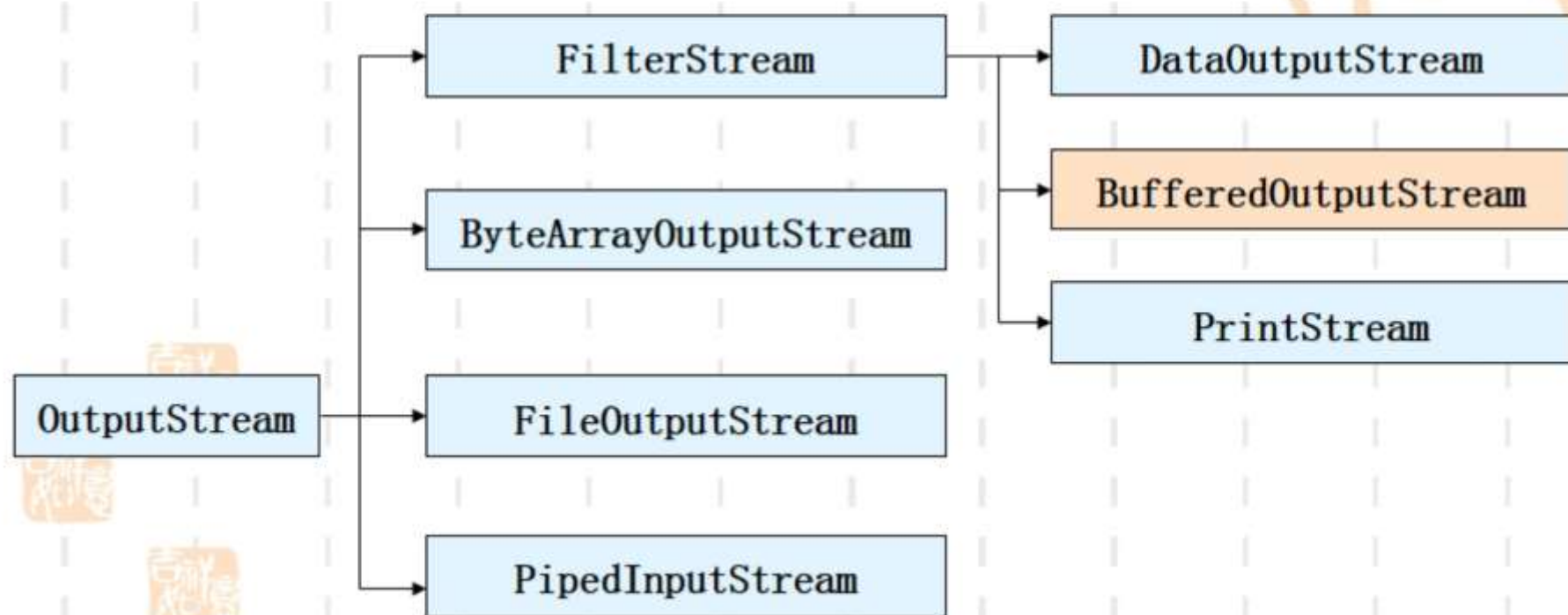
1. 字节输入流



类	描述
ByteArrayInputStream	以内存的一个缓冲区当作InputStream使用
StringBufferInputStream	将String 转化为InputStream
FileInputStream	读取文件中的信息
PipedInputStream	常用作多线程环境中的数据源
SequenceInputStream	将两个或多个InputStream转为单一的InputStream
FilterInputStream	是抽象类，作为其他装饰类的接口。
DataInputStream	和DataOutputStream协同应用，可从stream读入基本类型
BufferedInputStream	缓冲读，减少读次数
LineNumberInputStream	它会纪录stream内的行数。
PushbackInputStream	它会会送最后一个字节，一般用于编译器。



2. 字节输出流



类	描述
ByteArrayOutputStream	建立一块内存缓冲区，所有送至此Stream的数据都会被置于该缓冲区内
FileOutputStream	将信息写入文件
PipedOutputStream	常用作多线程环境中的数据源
FilterOutputStream	是抽象类，作为其他装饰类的接口。
DataInputStream	和DataInputStream协同应用，可将各种基本类型数据写至stream
BufferedInputStream	缓冲写，减少写次数，可以调用flush函数清出缓冲区内内容
PrintStream	产生格式化输出结果。



3.InputStream类方法

- public abstract int **read()** throws IOException
- public int read(byte[] b) throws IOException
- public int read(byte[] b, int off, int len) throws IOException
- public int **available()** throws IOException
- public void **close()** throws IOException



4. OutputStream 类方法

- public abstract void **write**(int b) throws IOException
- public void write(byte[] b) throws IOException
- public void write(byte[] b, int off, int len) throws IOException
- public void **flush**() throws IOException
- public void **close**() throws IOException





5. FileInputStream类

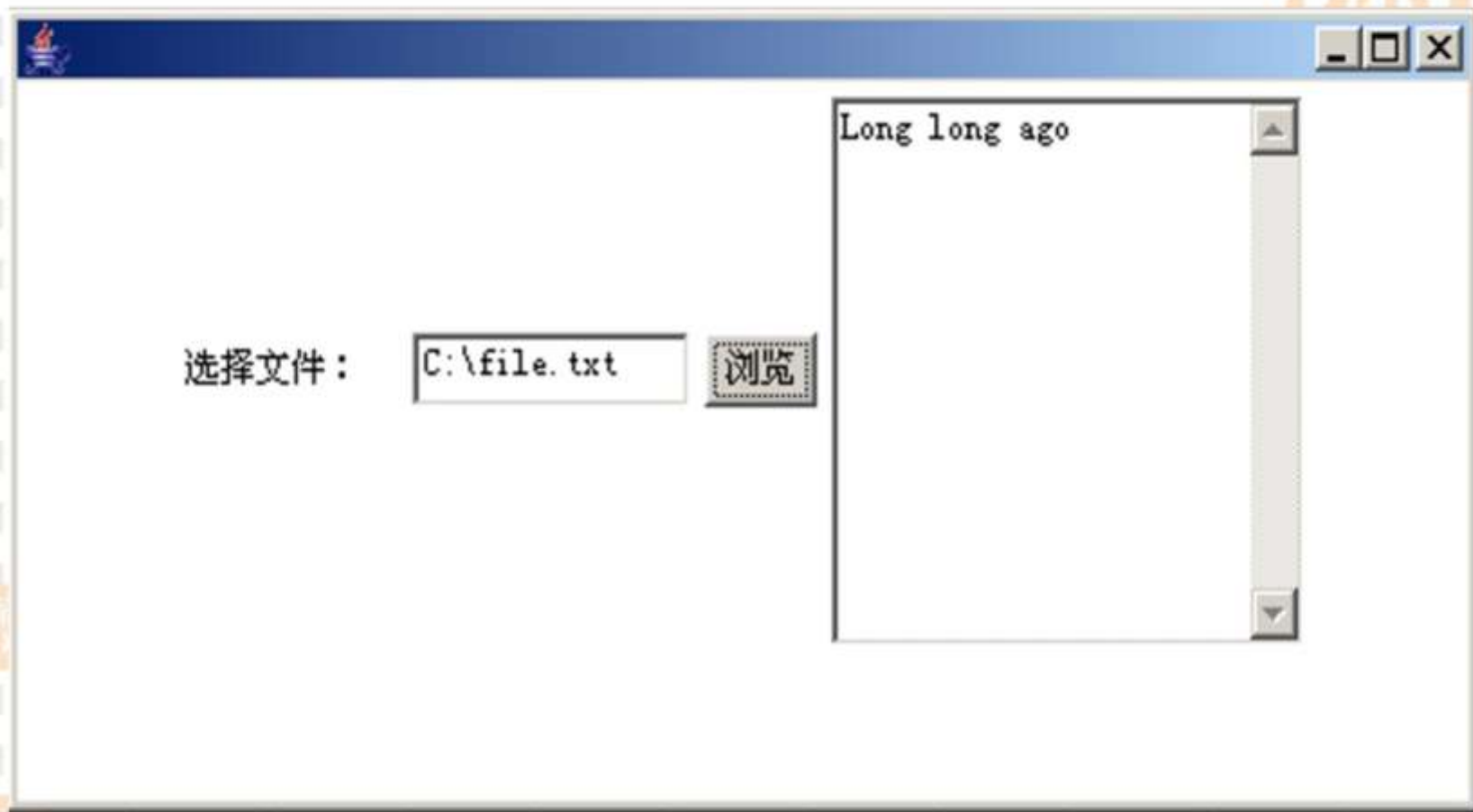
- public class FileInputStream extends
InputStream
- public FileInputStream(File file) throws
FileNotFoundException
- public FileInputStream(String name) throws
FileNotFoundException



6. **FileOutputStream**类

- public class FileOutputStream extends OutputStream
- public FileOutputStream(File file) throws FileNotFoundException
- public FileOutputStream(String name) throws FileNotFoundException
- public FileOutputStream(String name, boolean append) throws FileNotFoundException






```
import java.awt.*; import java.awt.event.*;
import java.io.*; import javax.swing.*;
public class test extends JFrame implements ActionListener{
    FileDialog m_fd;
    JTextField m_txtFilename=new JTextField(10);
    JLabel m_lblChoosefile=new JLabel("选择文件:");
    JButton m_btnChoose=new JButton("浏览");
    JTextArea m_txtareaFilecontent=new JTextArea(10,20);
    String m_sTemp;
    StringBuffer m_buf=new StringBuffer();
    test() {
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setLayout(new FlowLayout());
        add(m_lblChoosefile);
        add(m_txtFilename);
        add(m_btnChoose);
        add(m_txtareaFilecontent);
        m_btnChoose.addActionListener(this);
        setSize(500,250);
        setVisible(true);
    }
```



```

public void actionPerformed(ActionEvent e) {
    String cmd=e.getActionCommand();
    if(cmd=="浏览"){
        m_fd=new FileDialog(this,"打开文件",FileDialog.LOAD);
        m_fd.setDirectory("c:\\");
        m_fd.setVisible(true);

        m_sTemp=m_fd.getDirectory()+m_fd.getFile();

        m_txtFilename.setText(m_sTemp);
        try{
            BufferedInputStream bi=new BufferedInputStream(new
            FileInputStream(m_sTemp));

            while(bi.available()!=0){
                char a=(char)bi.read();
                m_buf.append(a);
            }
            bi.close();
            m_txtareaFilecontent.setText(m_buf.toString());
        }catch(Exception ee){}
    }
}

public static void main(String[] args){ new test(); }
}

```





```
public void actionPerformed(ActionEvent e) {
    String cmd=e.getActionCommand();
    if(cmd=="浏览"){
        m_fd=new FileDialog(this,"打开文件",FileDialog.LOAD);
        m_fd.setDirectory("c:\\");
        m_fd.setVisible(true);
        inFilename=m_fd.getDirectory()+m_fd.getFile();
        m_txtFilename.setText(inFilename);
        try{
            BufferedInputStream bi=new BufferedInputStream(new
                FileInputStream(inFilename));
            outFilename=m_fd.getDirectory()+"b.txt";
            BufferedOutputStream bo=new BufferedOutputStream(new
                FileOutputStream(outFilename));
            while(bi.available()!=0)
            { char a=(char)bi.read();
                m_buf.append(a);
                bo.write(a);}
            m_txtareaFilecontent.setText(m_buf.toString());
            bi.close();
            bo.close();
        }catch(Exception ee){ } }}

```



复习与预习

复习:

1. 流及字节流

预习:

字符流及文件



CITY INSTITUTE
城市学院